

Linear model

ANN:

↳ ANN are computational models inspired by the human brain, used to recognise pattern and solve problems like classification & prediction.

"ANN mimics how the human ~~learn~~^{brain} learns from data".

* basic idea:

In our brain

- ↳ Neurons receive signals
- ↳ Process them
- ↳ send output

ANN working

- ↳ Takes input
- ↳ Process it
- ↳ Produces output

* Structure of ANN:

↳ An ANN has layers of neurons.

1.) Input layer

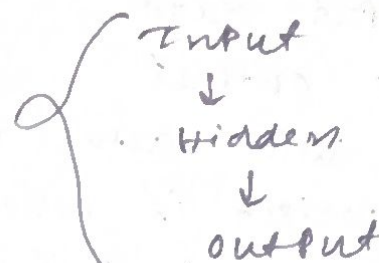
↳ Receive input

2.) Hidden layer (s)

- ↳ Process data
- ↳ learns pattern

3.) output layer

↳ Produces final result.



Data flow layer by layer.

* Key Concept:

• Neuron:

↳ Basic unit that process input.

• Weights:

↳ Importance given to each input.

• Activation funcⁿ

↳ Decides output of neuron

• Bias:

↳ Extra value to improve learning

* Advantages:

- ↳ Can solve complex problems
- ↳ learns non-linear relationship
- ↳ high accuracy

* Disadvantages:

- ↳ Requires large data
- ↳ computationally expensive
- ↳ hard to interpret.

* Application:

- ↳ image recognition
- ↳ speech recognition
- ↳ Recommendation system
- ↳ health sector.

Perceptron Learning Algorithm: (PLA)

↳ The PLA is a supervised learning algorithm used for binary classification, which learns a linear decision boundary to separate data into two classes.

"It classifies data into two classes using a linear boundary."

* Mathematical representation:

$$y = \begin{cases} 1 & \text{if } (w \cdot x + b) \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

this means:

↳ if value $\geq 0 \rightarrow$ class 1

↳ else $\rightarrow$ class 0

* How it works:

1. Initialize weight & bias
2. Take input data.
3. Calculate output
4. compare with actual output
5. update weight if wrong
6. Repeat until correct.

* Key point:

↳ It is only works for linearly separable data

↳ Like AND, OR.

↳ NOT for XOR.

* Adv:

- ↳ simple & fast
- ↳ easy to implement
- ↳ good for basic classification

* basic idea:

think of it as a simple neuron;

↳ takes input

↳ multiplies by weight

↳ Adds bias

↳ produces output (0 or 1)

* Weight update rule:

↳ if prediction is wrong:

• Adjust weight slightly

↳ Goal:

↳ Reduce error step by step

* AND Gate:

input	0, 0	$\rightarrow$	0
	0, 1	$\rightarrow$	0
	1, 0	$\rightarrow$	0
	1, 1	$\rightarrow$	1
	} $\rightarrow$ output		

↳ Perceptron can learn this pattern easily.

* disadv:

- ↳ can't solve non-linear problem
- ↳ only binary classification
- ↳ limited capability.

Single Layer Perceptron:

↳ A single layer perceptron is a type of artificial neural network that consists of only one layer of neurons (output layer) and is used for binary classification.

"It is a neural network with one layer that classifies data using a linear boundary"

* Basic idea

- ↳ takes input
- ↳ Applies weight
- ↳ produces output

* How it works:

- 1) Input values are given
- 2) multiply input with weight
- 3) Add bias
- 4) Apply activation function
- 5) produce output 0 or 1.

Key characteristics:

- ↳ only one layer
- ↳ no hidden layer
- ↳ linear decision boundary
- ↳ works for simple problems

* Advantages:

- ↳ simple and fast
- ↳ easy to implement
- ↳ low computation

Structure

- Input layer → receive data
- output layer → produces result

No hidden layer in between

Mathematical form:

$$y = f(w \cdot x + b)$$

where:

$w \cdot x$ = weighted sum

b = bias

f = activation function

example

↳ AND / OR Gate

can be solved using single layer perceptron.

↳ XOR can't be solved.

* Disadvantages:

- ↳ can't solve non-linear problems
- ↳ limited learning capacity.

Natural Language Processing (NLP)

↳ NLP is a field of ML that enables computers to understand, process, and analyze human language.

"NLP helps machines understand human language like text and speech".

NLP converts:

Text → Numerical form → Processing → Meaning

* Tokenization:

↳ Tokenization is the process of breaking text into smaller parts (tokens) like words or sentences.

ex: sentence: "learning AI is fun"

word level tokenization = ["learning", "AI", "is", "fun"]

subword level = ["learn", "ing", "AI", "is", "fun"]

character level = ["l", "e", "a", "r", "n", "i", "n", "g", "A", "I", "i", "s", "f", "u", "n"]

* Application of NLP:

- ↳ chatbots
- ↳ language translation
- ↳ Sentiments analysis
- ↳ Speech recognition

* Advantages:

- ↳ Automate language tasks
- ↳ Saves time
- ↳ handles large text data

* Disadvantages:

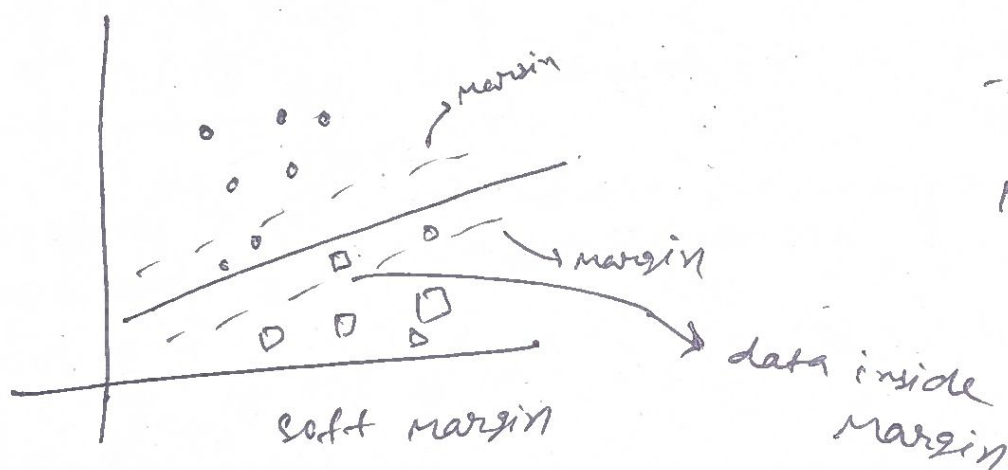
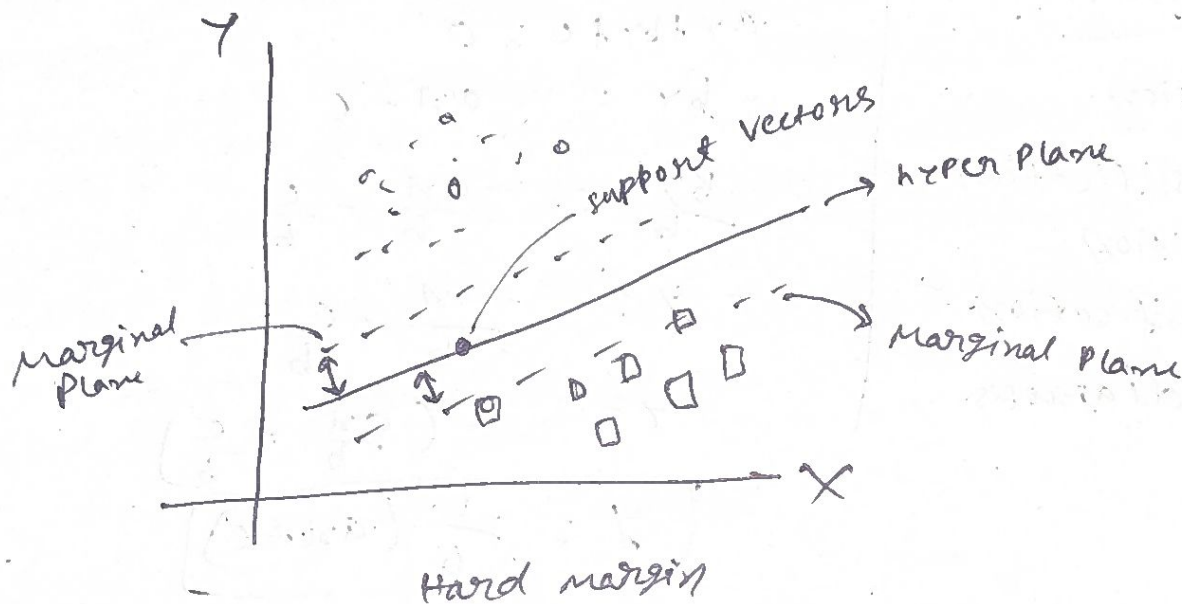
- ↳ Language Ambiguity
- ↳ Context understanding is difficult
- ↳ Require large data

Module - 3

SVM:

↳ Support vector machine (SVM) is a supervised ML algorithm used mainly for classifications, which finds the best boundary (hyperplane) that separates data into different classes.

"SVM finds the optimal separating boundary between classes".



- → hyperplane
(decision boundary)
 - - - → Marginal plane
(parallel to hyperplane)
 - ↔ → margin
(distance b/w marginal planes)
 - → support vectors
(points on marginal planes).
- Support vectors are the nearest data point to hyperplane

How SVM works:

- 1) Take Training data
- 2) find possible separating hyperplanes
- 3) calculate margin for each
- 4) select hyperplane with max^m margin
- 5) use it for classification.

Why SVM:

- ↳ It focuses on
 - max^m separation
 - better generalization
 - robust classification

* Advantages:

↳ high accuracy

↳ effective in high-dimensional data

↳ works well for classification

↳ Reduces overfitting using max^m margin

* disadv:

↳ slow for very large datasets

↳ hard to interpret

↳ choosing kernel can be difficult.

* Application of SVM:

↳ Spam detection

↳ Image classification

↳ face recognition

↳ Text classification

↳ medical diagnosis.

$$y = mx + c$$

$$ax + by + c = 0$$

$$ax + by + c = 0$$

$$by = -ax - c$$

$$\frac{by}{b} = \frac{-ax}{b} - \frac{c}{b}$$

$$y = \frac{-ax}{b} - \frac{c}{b}$$

$$y = -\left(\frac{ax}{b} + \frac{c}{b}\right)$$

$$y = -\frac{1}{b}(ax + c)$$

* K-means Clustering:

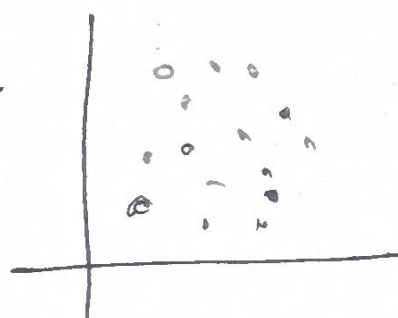
↳ K-mean is an supervised learning algorithm used to group data into k -clusters, where each data point belongs to the cluster with the nearest mean (centroid).

"It divides data into k -groups based on similarity."

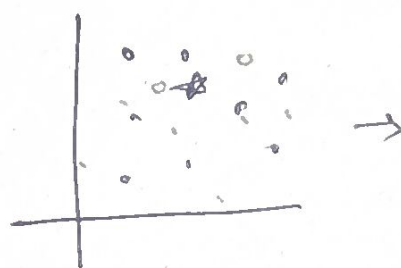
* How it works:

1. Choose number of clusters (k)
2. Initialize k centroid (random points)
3. Assign each data point to nearest centroid
4. Recalculates centroids (mean of cluster)
5. Repeat steps 3-4 until no change

this is called iteration.



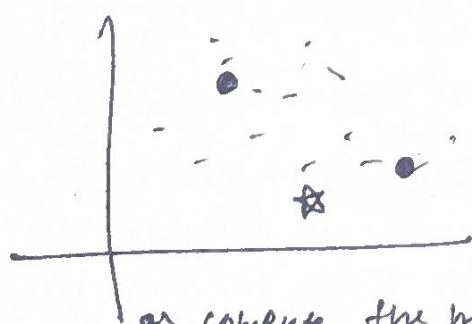
1. Choose k
Let's say $k=3$



2. Initialize centroids
Randomly choose k



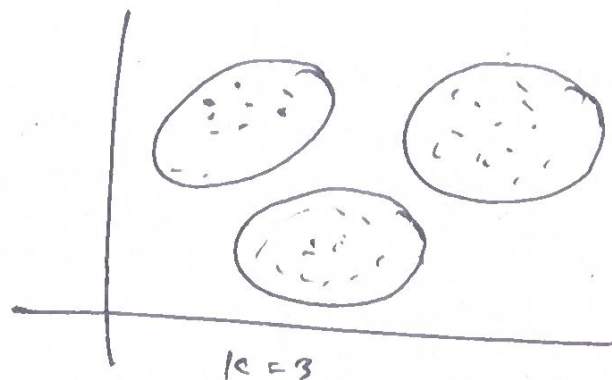
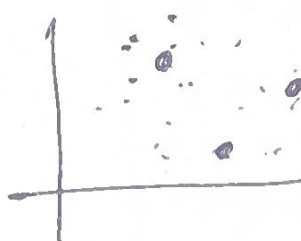
3. Assign each data point to the nearest centroid.



4. Compute the mean of each cluster and update centroids.

→ Recalculate centroids

5. Repeat



$k=3$

Key Points:

↳ K-means is an iterative algo.

↳ It tries to minimize the distance b/w points and centroids,

↳ center of cluster: calculated as mean of all clusters.

Choosing k

- Too small k = poor grouping
- Too large k = overfitting

* Advantages:

- ↳ simple and fast
- ↳ easy to implement
- ↳ works well for large datasets.

* Disadvantages:

- ↳ Need to choose k manually
- ↳ sensitive to initial centroid
- ↳ Not good for complex shapes.

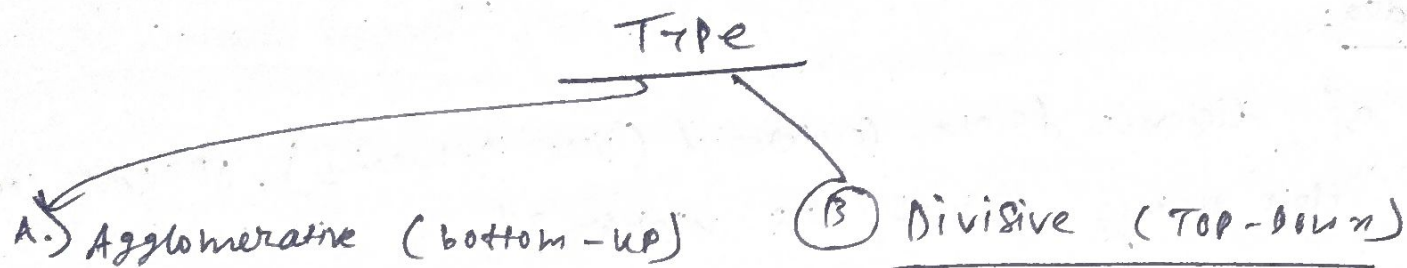
* Application:

- ↳ customer segmentation.
- ↳ image compression.
- ↳ pattern recognition.

Module - IV

Hierarchical clustering:

↳ Hierarchical clustering is an unsupervised learning algorithm that groups data into a hierarchy of clusters, represented as a tree-like structure (dendrogram).



→ Start with each data point as a separate cluster

→ Merge the closest clusters repeatedly

→ Stop when all points become one cluster

How it works:

- 1) Treat each data point as a cluster
- 2) Calculate distance b/w clusters.
- 3) Merge closest clusters.
- 4) Update distances.
- 5) Repeat until one cluster remains.

Applications:

- ↳ Biology (Gene Clustering)
- ↳ Document grouping
- ↳ Customer segmentation

Distance b/w clusters

- single linkage: → nearest points
- complete linkage: farthest points
- Avg linkage: avg. distance

* Advantages:

- ↳ No need to choose k initially
- ↳ Easy to understand (tree structure)
- ↳ works well for small datasets

* Disadv:

- ↳ slow for large data
- ↳ can't undo merging
- ↳ sensitive to noise

DBSCAN:

↳ DBSCAN is an unsupervised learning algorithm that groups data points based on density.

ex → areas where points are closely packed together.

"It forms clusters based on dense region and detects noise".

* Basic idea:

→ instead of distance from centroid (like k-means), DBSCAN look at: "How many points are nearby?"

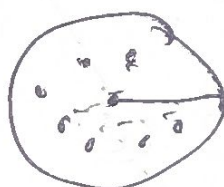
→ if many points are close → cluster

→ if isolated → noise (outlier)

* Key Parameter:

ϵ (epsilon)

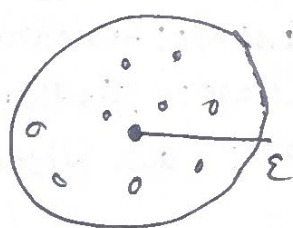
Is the radius of the neighbourhood



minpts:

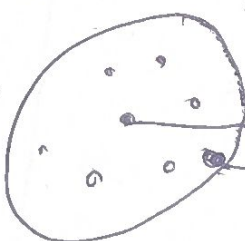
min^m number of points required within ϵ -neighbourhood to form a dense region (core point)

Core Point



minpts = 5

Borderpoint

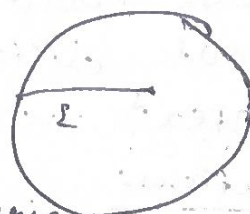


minpts = 5

A point with fewer than minpts neighbours, but within ϵ of a core point.

→ A core point at least minpts neighbour within ϵ

Noise Outlier



A point with fewer than minpts neighbour and not within ϵ of any core point.



How it works:

- 1.) Choose ϵ and minpts
- 2.) Pick a point
- 3.) Check neighbours within ϵ
- 4.) If neighbours $\geq$ minpts $\rightarrow$ form cluster
- 5.) Expand cluster
- 6.) Repeat for all points

Key features

- $\hookrightarrow$ DBSCAN can:
- $\hookrightarrow$ detect arbitrary shapes (not just circles)
- $\hookrightarrow$ identify noise automatically.

* Advantages:

- $\hookrightarrow$ No need to choose k
- $\hookrightarrow$ Detects noise/outliers
- $\hookrightarrow$ Works for complex shapes

* Disadvantages:

- $\hookrightarrow$ Choosing ϵ is tricky
- $\hookrightarrow$ Not good for varying densities
- $\hookrightarrow$ Slower for large datasets.

* Applications:

- $\hookrightarrow$ Anomaly detection
- $\hookrightarrow$ Image processing
- $\hookrightarrow$ Spatial data analysis

Clustering Validation:

↳ Clustering validation measures are used to evaluate the quality of clusters formed by clustering algorithms.

"They check how good the clustering is".

* Basic idea:

↳ A good clustering should have:

- Points in same cluster → close to each other
- Points in different cluster → far from each other.

Types of Validation

1. Internal measures

↳ evaluate clustering using only dataset (no labels)

a) Intra-cluster Distance:

↳ Distance between points within same cluster
↳ should be small

b) Inter-cluster Distance:

↳ distance b/w different clusters
↳ should be large.

Good clustering = low intra-cluster ↓
high inter-cluster ↑

2. External clusters:

↳ compare clusters with true labels (if available)

ex: Accuracy, Precision, recall.

3. Relative measure:

↳ compare different clustering result.

ex: which k is better in k-means.